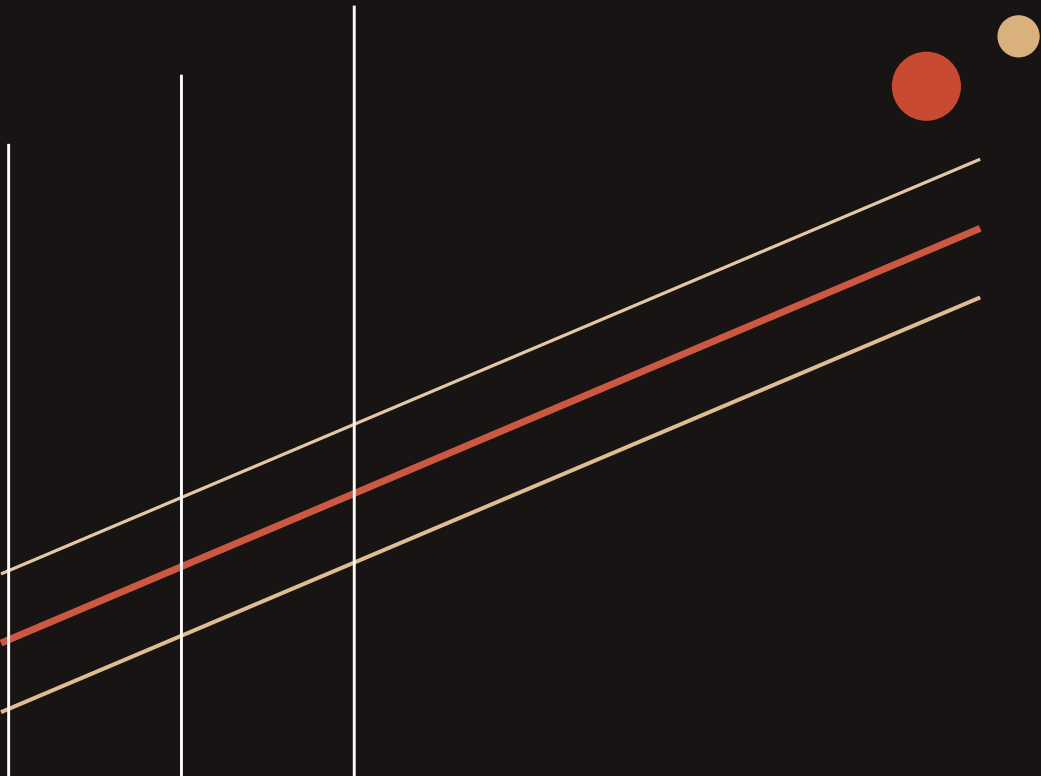


VIBE CODING IN THE AI ERA

Vibe Coding Tauri 2

Sample Edition · 요청·구현·검증—AI와 반복으로 짓는 4개 데스크톱 앱

Request | Build | Verify | Repeat



Ted Park

2026

Vibe Coding Tauri 2

Sample Edition · 요청·구현·검증—AI와 반복으로 짓는 4개 데스크톱 앱

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

차례

1 이 책에 대하여	1
1.1 Why Vibe Coding	1
1.2 이 책이 다루는 것 / 다루지 않는 것	2
1.3 핵심 루프—요청·구현·검증	2
1.4 4개 앱 소개	2
2 전체 여정 지도	4
3 챕터별 요약	5
3.1 1장. 왜 AI로 개발하는가	5
3.2 2장. 요청·구현·검증	5
3.3 3장. 도구와 환경	6
3.4 4장. Mandai 설계—앱 한 덩이를 AI에 어떻게 던질까	6
3.5 5장. DB 스키마—첫 루프의 교본	6
3.6 6장. AI API 통합—AI로 AI 코드 짜기	6
3.7 7장. Mandai의 함정—AI가 틀린 것들	6
3.8 8장. readbooks.ai —프롬프트 설계	7
3.9 9장. 번역 검증—AI 출력의 신뢰성을 어떻게 확인하나	7
3.10 10장. Trading Dashboard —AI가 만들 수 있는 규모의 한계	7
3.11 11장. WebSocket —AI가 자주 틀리는 영역	7
3.12 12장. 배포—Cloudflare Tunnel과 sidecar를 AI로	7
3.13 13장. RL & Backtest Lab —AI가 모르는 도메인 지식	8
3.14 14장. 당신의 앱에 이 워크플로우 적용하기	8
4 본문 미리보기—2장. 요청·구현·검증	9

4.1 루프의 네 단계	9
4.2 Mandai DuckDB 스키마—두 사이클	10
4.3 왜 루프인가	10
5 이 책을 마치면	12

이 책에 대하여

1.1 Why Vibe Coding

나는 시간이 많은 개발자가 아니다. 회사에 다니고, 아이를 키운다. 평일 저녁 한두 시간, 주말 몇 시간이 개발에 쓸 수 있는 전부다. 예전이라면 이 시간으로 앱 하나도 벅찼다.

그런데 2026년 지금, 나는 Tauri 2 기반 데스크톱 앱을 **네 개** 만들어 돌리고 있다. 기술 스택도 쉬운 게 아니다. Rust 백엔드, SvelteKit 프론트, DuckDB, WebSocket, Python sidecar, Cloudflare Tunnel.

차이는 하나—**AI 에이전트**다. Claude Code에 맥락을 주면 저녁 30분 짬에 DB 스키마가 나온다. 다음 날 출근 전 15분에 버그가 고쳐진다. 에이전트가 타이핑을 하고, 나는 요청하고 검증한다. **시간이 많아 서가 아니라, 시간이 없는데도 AI 에이전트 덕분에 네 개를 만들 수 있었다.**

이 책은 그 방식을 다룬다. 코드 튜토리얼이 아니다. Rust 문법·Svelte API 레퍼런스가 아니다. 대신 **AI 에이전트와 어떻게 일하는가**를 네 개 앱 케이스로 보여준다.

1.2 이 책이 다루는 것 / 다루지 않는 것

다룬다	다루지 않는다
AI 에이전트에게 요청하는 법	Rust · Svelte 문법 튜토리얼
구현 결과 검증하는 루틴	Tauri API 레퍼런스
루프 반복의 패턴	최신 라이브러리 목록
실패한 프롬프트와 이유	“이 한 가지 방법”식 마법 공식
네 개 실전 앱의 설계·검증 기록	10줄 넘는 코드 덩어리

1.3 핵심 루프—요청·구현·검증

이 책 전체를 관통하는 네 단계.

단계	뜻	나의 역할
요청 (Request)	맥락·제약·목표를 에이전트에 전달	문제를 정의한다
구현 (Build)	에이전트의 산출물 (코드·설계·문서)	기다린다
검증 (Verify)	실행·테스트·리뷰	결함을 찾는다
반복 (Repeat)	틀린 부분을 다시 요청	루프를 닫는다

한 번의 프롬프트로 끝나는 기적은 없다. **루프 횟수를 줄이는 기술**이 있을 뿐이다.

1.4 4개 앱 소개

앱	역할	드러난 AI의 면
Mandai	만다라트 + GTD + 뽀모도로 + 멀티 AI	DuckDB 스키마, Tauri IPC, SSE 파싱은 AI가 잘 안다
readbooks.ai	PDF 36개 언어 번역	프롬프트 자체를 코드처럼 관리
Trading Dashboard	19 화면 실시간 트레이딩	대규모 앱 쪼개기의 한계
RL & Backtest Lab	Python 프로세스 스폰, 실시간 로그	도메인 지식을 에이전트에 주입하는 법

CHAPTER 2

전체 여정 지도

Part	챕터	주제
I	1 -3장	왜 AI ·루프·도구
II	4 -7장	Mandai —설계·DB ·AI API ·함정
III	8 -9장	readbooks.ai —프롬프트·검증
IV	10 -12장	Trading Dashboard —규모·실시간·배포
V	13장	RL & Backtest Lab —도메인 지식 주입
VI	14장	당신의 앱에 적용하기

챕터별 요약

3.1 1장. 왜 AI로 개발하는가

2019년의 나는 앱 하나에 6개월이 걸렸다. 2026년의 나는 네 개를 몇 달에 만들었다. 시간이 늘어난 게 아니다—오히려 줄었다 (육아 중). 타이핑을 에이전트에게 위임하며 역할이 **코드 쓰기**에서 **프롬프트 쓰기**로 이동했다. 모델 성숙·Tauri 2 GA·에이전트 도구 세 흐름이 왜 지금 겹쳤는지 짚는다.

3.2 2장. 요청·구현·검증

책 전체를 관통하는 루프를 정의한다. Mandai DuckDB 스키마 두 사이클을 실제로 돌려보며 What/Why/Constraints/Format 프롬프트 템플릿을 손에 익힌다. “한 번의 프롬프트”망상을 깨고, 검증 없는 구현이 왜 기술 부채가 되는지 말한다.

3.3 3장. 도구와 환경

Claude Code · Cursor · Codex 세 도구의 철학 차이. 작업 유형별 선택 기준. MCP 서버 · Agent · Slash Command를 루프 단축 장치로 재해석. CLAUDE.md · .cursorrules 실제 발췌. 4개 앱 각각에 어느 도구를 어떻게 배분했는지.

3.4 4장. Mandai 설계—앱 한 덩이를 AI에 어떻게 던질까

“다 짜줘”의 실패. 에이전트가 되물을 수 있도록 맥락을 쪼개는 기술. 셀 ID · 뽀모도로 귀속 · 오늘 화면 세 가지를 각각 루프로 돌려 완성한 과정.

3.5 5장. DB 스키마—첫 루프의 교본

DuckDB 스키마를 3왕복에 완성한 기록. 한 번에 맞힌 코드(마이그레이션 러너, 커맨드 라우팅, TS 타입 레이어)와 루프로 맞춘 코드(스키마 자체)의 비율을 비교한다.

3.6 6장. AI API 통합—AI로 AI 코드 짜기

OpenAI · Claude · Gemini · Groq 4개 프로바이더 스트리밍 통합. Tauri emit 빼대는 한 번에, Anthropic SSE event: 분기는 3왕복. 에이전트의 강·약 영역 지도.

3.7 7장. Mandai의 합정—AI가 틀린 것들

뽀모도로 타이머 동기화 5왕복. 만다라트 셀 ID 추측 오류. 생산성 계산 엣지 케이스 5왕복. “읽을 줄 알아야 하는”바이브 코딩의 역설.

3.8 8장. readbooks.ai —프롬프트 설계

제약 박기 -> 언어별 프로파일 -> 에이전트를 전문가·리뷰어로 재사용하는 메타 루프. 프롬프트를 .md 파일로 코드처럼 관리한다.

3.9 9장. 번역 검증—AI 출력의 신뢰성을 어떻게 확인하 나

역번역 + 3축 스코어링(의미·길이·용어). 3단계 배지 임계치. 사용자 피드백을 프롬프트에 반영. Cloudflare Workers ReadableStream 캐시 이슈를 Claude와 디버깅한 실전 기록.

3.10 10장. Trading Dashboard —AI가 만들 수 있는 규 모의 한계

19 화면 실시간 대시보드. 큰 앱을 AI-가능한 단위로 쪼개는 기술. 화면·컴포넌트·데이터 단위 분할. 글로벌 상태(tradingStore) 설계가 에이전트에게 왜 어려웠나. (스크린샷 Figure 10.1 포함)

3.11 11장. WebSocket —AI가 자주 틀리는 영역

에이전트가 처음 내놓은 코드의 세 구멍: 재연결 없음, 토큰 매번 요청, 포트 하드코딩. 지수 백오프·thundering herd·jitter 같은 도메인 용어를 에이전트에 전달하는 방법. AtomicU16 3왕복.

3.12 12장. 배포—Cloudflare Tunnel과 sidecar를 AI로

Tauri 배포의 세 복잡성—.app 번들, sidecar 경로, 원격 접속. Cloudflare Tunnel 자동화는 한 번에 성공, sidecar 경로는 3왕복. is_remote() 분기는 사람이 먼저 설계한다.

3.13 13장. RL & Backtest Lab —AI가 모르는 도메인 지식

Tokio subprocess는 에이전트가 잘 안다. RL 훈련 실패 패턴·backtest 검증 기준은 모른다. 도메인 지식을 .md 노트로 주입하는 방법. RESULT_JSON: 프리픽스 다중화—AI 훈련 데이터에 없던 나만의 트릭.

3.14 14장. 당신의 앱에 이 워크플로우 적용하기

아이디어 -> 화면 -> 데이터 -> 커맨드 순서. 첫 프롬프트 템플릿(What ·Why ·Constraints ·Format). 실패 패턴 5가지와 대응. 프롬프트를 라이브러리로 만드는 법. “나와 같은 조건—바쁜 직장인·육아 중·시간 부족”에서도 가능하다.

CHAPTER 4

본문 미리보기—2장. 요청·구현·검증

이 책의 핵심 루프가 가장 명확히 드러나는 장. 실제 본문에서 발췌.

4.1 루프의 네 단계

단계	뜻	핵심 질문
요청	맥락·제약·목표를 에이전트에 전달	“에이전트가 이것만 보고 시작할 수 있는가?”
구현	에이전트의 산출물	—

단계	뜻	핵심 질문
검증	실행·테스트·리뷰	“이 결과가 내가 원하는 것과 같은가?”
반복	틀린 부분을 다시 요청	“어느 부분이 왜 틀렸는가?”

4.2 Mandai DuckDB 스키마—두 사이클

사이클 1—첫 요청

DuckDB 스키마를 짜줘. 만다라트 9×9 셀 + 태스크 + 뽀모도로 세션. 외래 키로 연결.

Claude의 응답:

```
CREATE TABLE mandala_cells (id TEXT PRIMARY KEY, label TEXT);
CREATE TABLE tasks (id TEXT PRIMARY KEY, cell_id TEXT REFERENCES
mandala_cells(id));
CREATE TABLE pomodoro_sessions (id TEXT, task_id TEXT, start_at
TIMESTAMP);
```

실행해봤다. 세션 총 집중 시간을 계산하려 해보니 `end_at`이 없다. 인덱스도 없다. 소프트 삭제 칼럼도 없다.

사이클 2—다시 요청

`end_at` `TIMESTAMP`, 각 테이블에 `created_at·updated_at·deleted_at`. `pomodoro_sessions.task_id`에 인덱스.

한 번에 맞췄다. 총 두 번의 루프, 약 5분. 손으로 짰으면 30분.

4.3 왜 루프인가

한 번의 완벽한 프롬프트를 추구하면 프롬프트 자체가 길어진다. 100줄짜리 프롬프트는 에이전트가 어느 요구를 놓쳤는지 추적이 어렵다. **짧은 프롬프트 × 검증 × 짧은 수정 요청** 조합이 유지보수 가능한 루프다.

루프의 속도는 **검증 속도**가 결정한다. 스키마는 즉시 실행 가능하니 빠르다. UI 렌더링은 눈으로 봐야 하니 느리다. 알고리즘은 테스트 케이스가 있어야 하니 중간이다. 각 영역의 검증 방식에 맞춰 프롬프트 크기를 조정한다.

이 책을 마치면

1. **AI 에이전트에게 요청을 잘 쓰는 법**을 안다. 맥락·제약·목표·형식 네 요소를 담는 법.
2. **구현 결과를 검증하는 루틴**을 가진다. 실행·테스트·리뷰·반영.
3. **AI 강·약 영역 지도**를 그린다. 웹 UI 90% vs 신설계 영역 30% 같은 감각.
4. **당신의 앱에 같은 루프를 적용**할 수 있다. Mandai가 아니어도, Tauri가 아니어도, **바쁜 조건에서도**.
5. **실패 패턴 5가지에 대응**할 수 있다. 그럴듯한 오답, 반복 실수, 과도한 복잡도, 테스트 생략, 구버전 API 혼동.